

Modelling process :

Data Collection → Database, opensource datasets, kaggle ...

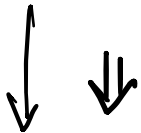


EDA (Understand distributions) ⇒ Continuous processes

EDA 1
EDA 2
EDA 3
⋮



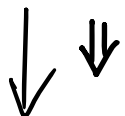
Feature Engineering ⇒ Continuous process



Transformations



Algorithm Selection



Hyperparameter Tuning



Model Evaluation

Model

Multiple versions of Models /
file / features / parameters



Each step influences your final Model / your approach

You will try many different approaches / experiments

How do you keep a track?

For example,

feature A, B, C with hyperparameter tuning of α, β, γ
gave accuracy of 92%.

feature B, C with hyperparameter tuning of γ
gave accuracy of 83%.

Modifying model to Random Forest improved accuracy to 94%.

It is difficult to keep track of these manually

Solution : MLFlow ^{free}



MLflow is like Git + Google Sheets + Notes for machine learning experiments.

files → numbers → notes
Models
images

Nice Simple UI

Typical situation where MLflow helps

You train:

- XGBoost with 10 parameter sets
- Random Forest with 5 variants
- Logistic Regression with feature tweaks

Without MLflow → confusion

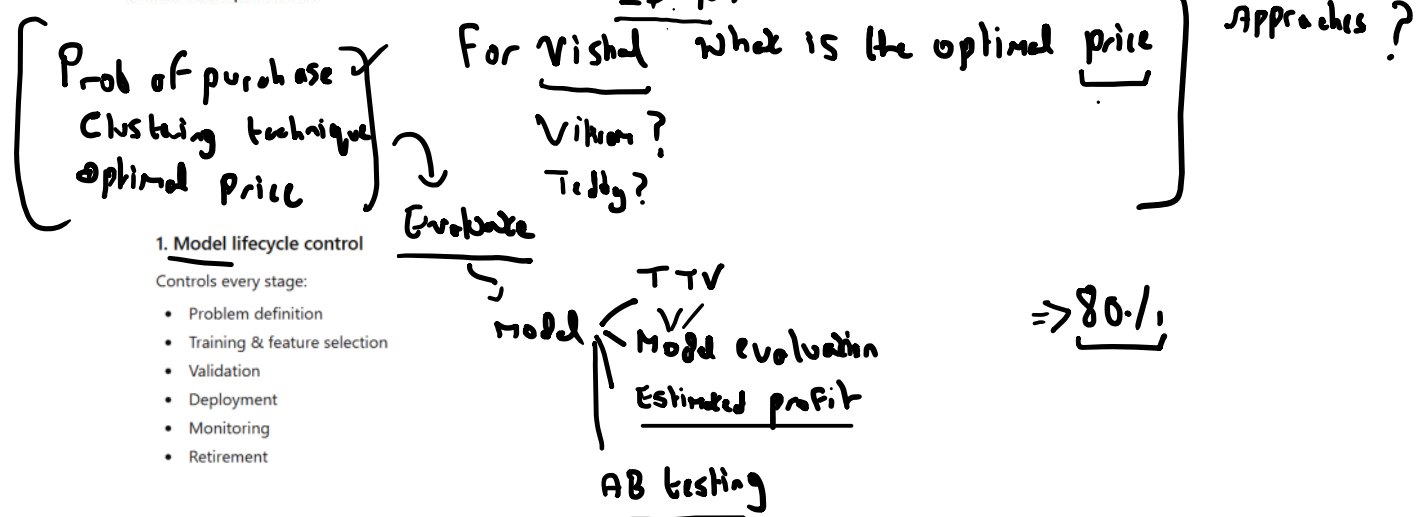
With MLflow → clear winner, clean history, less stress

ML system Design

AI
Agent

features Targets

Model governance in machine learning is the set of processes, controls, and accountability mechanisms that ensure ML models are built, deployed, and used responsibly, reliably, and in line with business, legal, and ethical requirements.



2. Versioning & reproducibility

Ensures you can recreate any model decision.

Includes:

- Model versions
- Training data snapshot / hash
- Feature definitions
- Code + hyperparameters

Example

"Why did the price prediction change on 12th Dec?"

→ Governance lets you trace it to model v3.2 trained on data till Nov 30

3. Validation & risk management

Formal checks before deployment:

- Performance (train vs validation vs OOT)
- Stability (PSI, CSI)
- Bias & fairness
- Stress testing

4. Explainability & transparency

Ability to explain:

- Global behavior (feature importance, PDPs)
- Local decisions (SHAP, LIME)

Governance requirement

A model that can't be explained may be banned from high-risk use cases (credit, pricing, hiring).

5. Monitoring & drift detection

After deployment:

- Input drift
- Prediction drift
- Metric decay
- Data quality failures

Example

- Mileage distribution shifts → resale price model becomes unreliable
- Governance mandates retraining or rollback

Model Governance

[Experimentation + Evaluation should be accounted for at all stages of your Model lifecycle]



Auditing every version of your Model/approach/data... -..

What does model governance involve in a machine learning project?

31 users have participated

A

Only deploying the best model for user access

3%

B

Ensuring that project code is well documented

6%

C

Tracking the experiment from the experimentation to deployment for auditing purposes

90%

D

Reducing the data required for training the model

0%

small part

From <<https://www.scaler.com/instructor/meetings/i/experiment-tracking-data-management-using-mflow-53/>>

Why is it important to keep track of data in machine learning projects?

32 users have participated

A

(Data is the code that makes machine learning models work) ✓

6%

B

Data can be sourced from multiple storage units ✓

6%

C

Data helps in analyzing features at different steps of the project ✓

16%

D

All of the above

72%

(From <<https://www.scalex.com/instructor/meetings/l/experiment-tracking-data-management-using-mlflow-53/>>

Our use case

04 January 2026 12:53

Basic mlflow run

04 January 2026 12:53

Other important sections

04 January 2026 18:30

OverviewModel metricsSystem metricsTracesArtifacts

Description

No description

Metrics (1)

Search metrics

Metric	Value	Models
mape	0.23136192560195923	model

Parameters (3)

Search parameters

Parameter	Value
-----------	-------

About this run

Created at

01/04/2026, 06:25:21 PM

Created by

swath

Experiment ID

1

Status

Finished

Run ID

da1e1bfed60149bda7c247ec161ad7d6

Duration

4.4s

Source

C:\Users\swath\AppData\Roaming\Python\Python314\site-packages\ipykernel_launcher.py

Registered prompts

Datasets

None

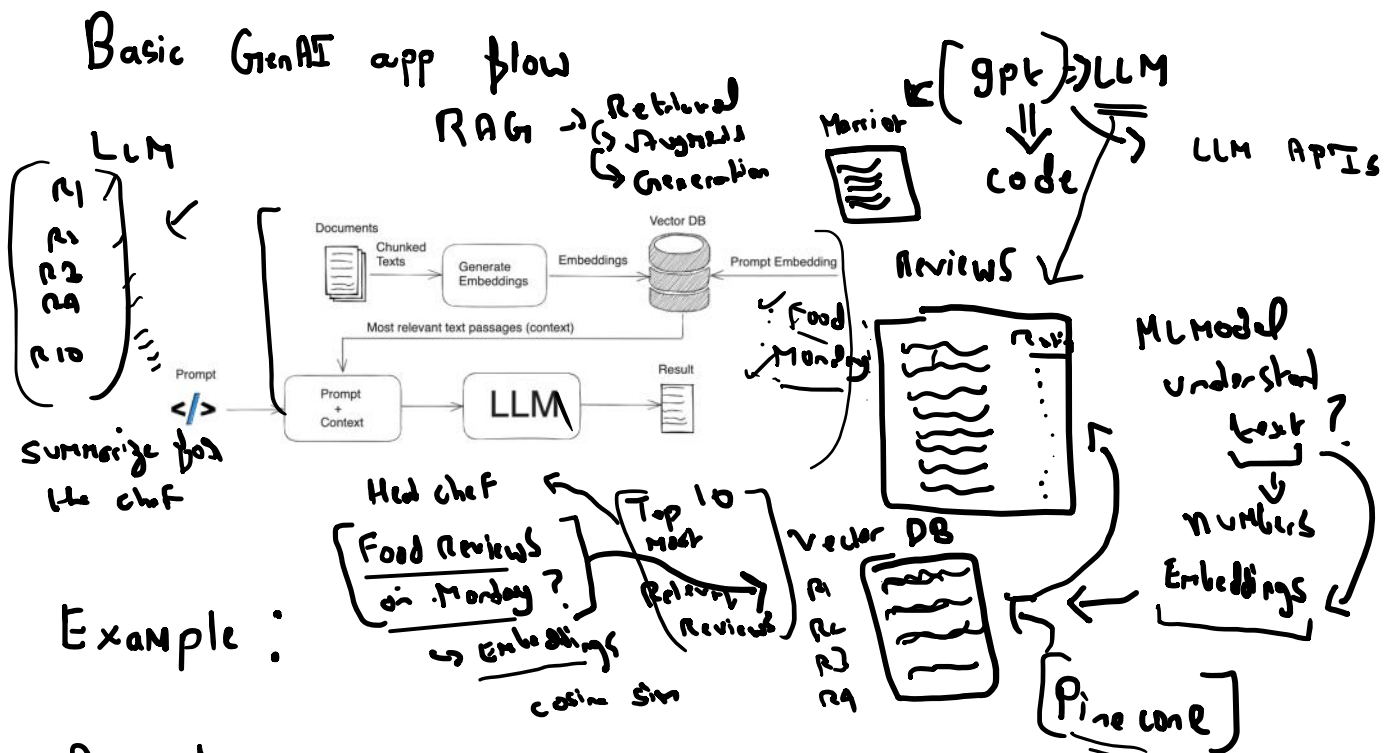
Tags

Add tags

One-line definition

Traces in MLflow record step-by-step execution of an application, not model training.

New feature introduced after emergence of GenAI (LLMOps)



App to assist different departments in a popular hotel chain.

[Hundreds of customer reviews + ratings every month]

Based on reviews + ratings, give performance summary for:

Food and Bev Manager

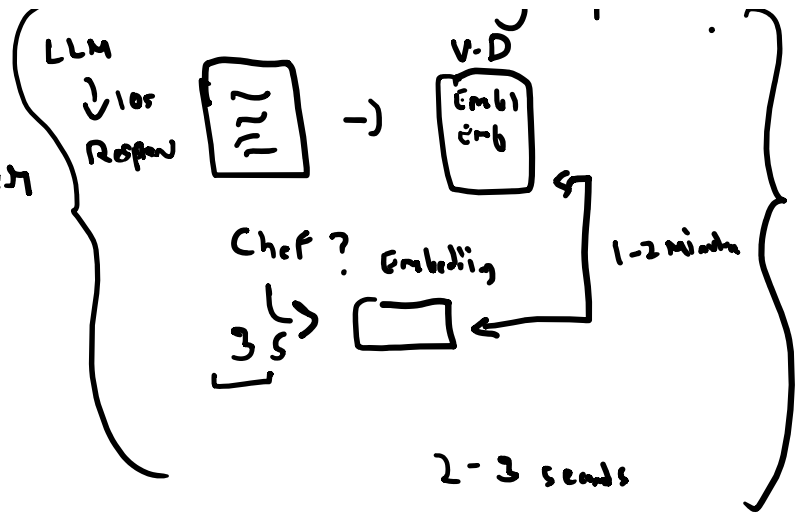
LLM .1.10s [] . [V-D Embed]

Food and Bev Manager

Housing and Accommodation Manager

Pool and Garden Manager

⋮



Overall summary for Hotel Manager

Role of traces :

Imagine an LLM app:

1. User query received
2. Vector search called
3. Prompt constructed
4. LLM called
5. Post-processing applied

A trace records:

- Each step
- Start / end time
- Inputs & outputs
- Errors (if any)

This is **debugging** + **observability**, not evaluation.

Why MLflow added traces

Classic ML:

- Train once
- Evaluate offline

LLM apps:

- Run continuously
- Many steps
- Hard to debug
- Latency matters

Why this is powerful (real problems it solves)

1. Debug bad answers

You can see:

- Did retrieval return junk?
- Was prompt malformed?

2. Latency optimization

You instantly know:

- LLM call = 1.8s bottleneck
- Retrieval = fine

Now you know **where** to optimize.